

STA 5635 HW 8

Palmer Swanson

March 5, 2026

Part a

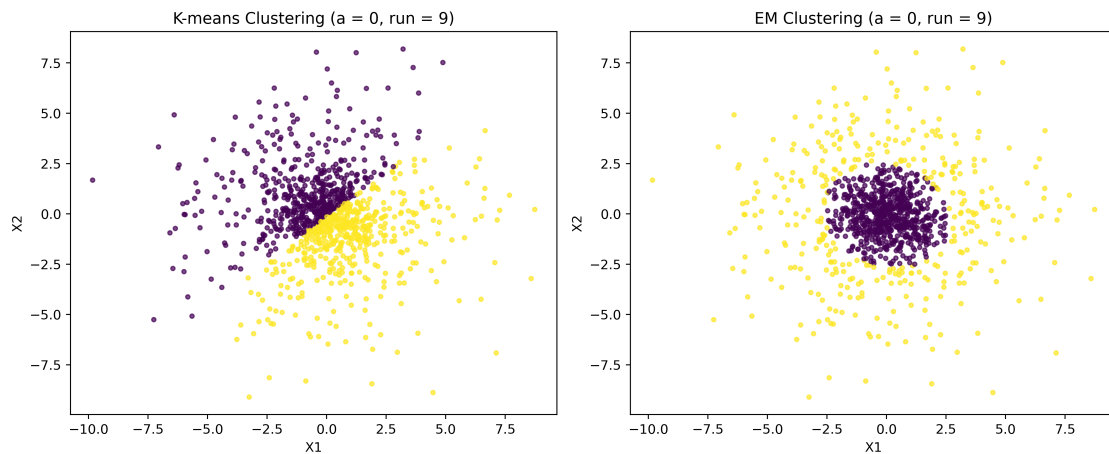


Figure 1: Clustering Results for K-means and EM, $a=0$

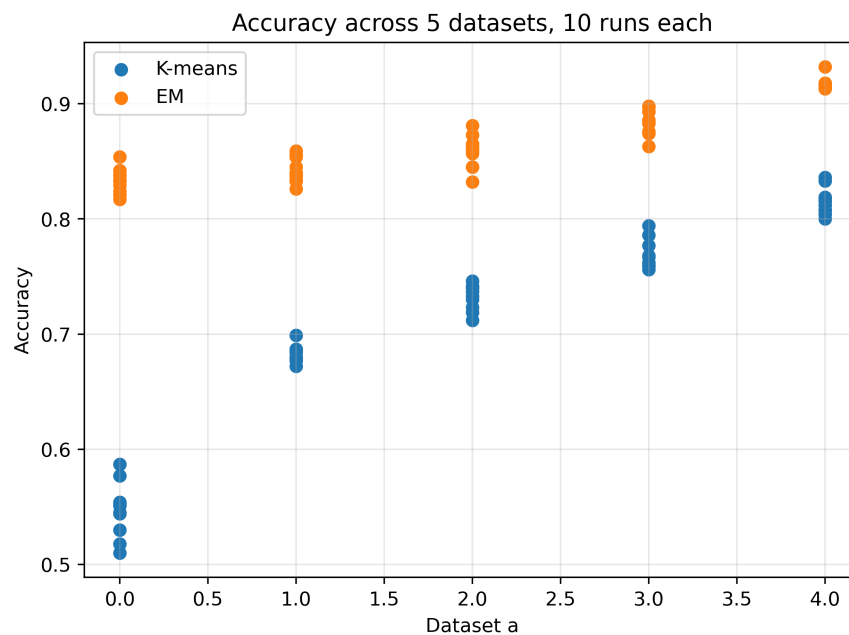


Figure 2: Accuracy for each run vs. dataset parameter a

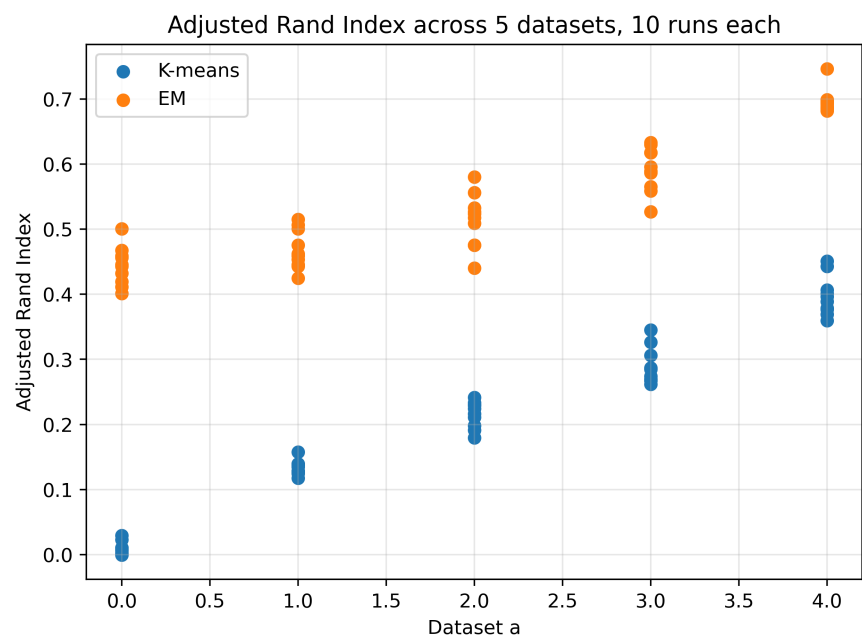


Figure 3: Adjusted Rand Index for each run vs. dataset parameter a

Part b

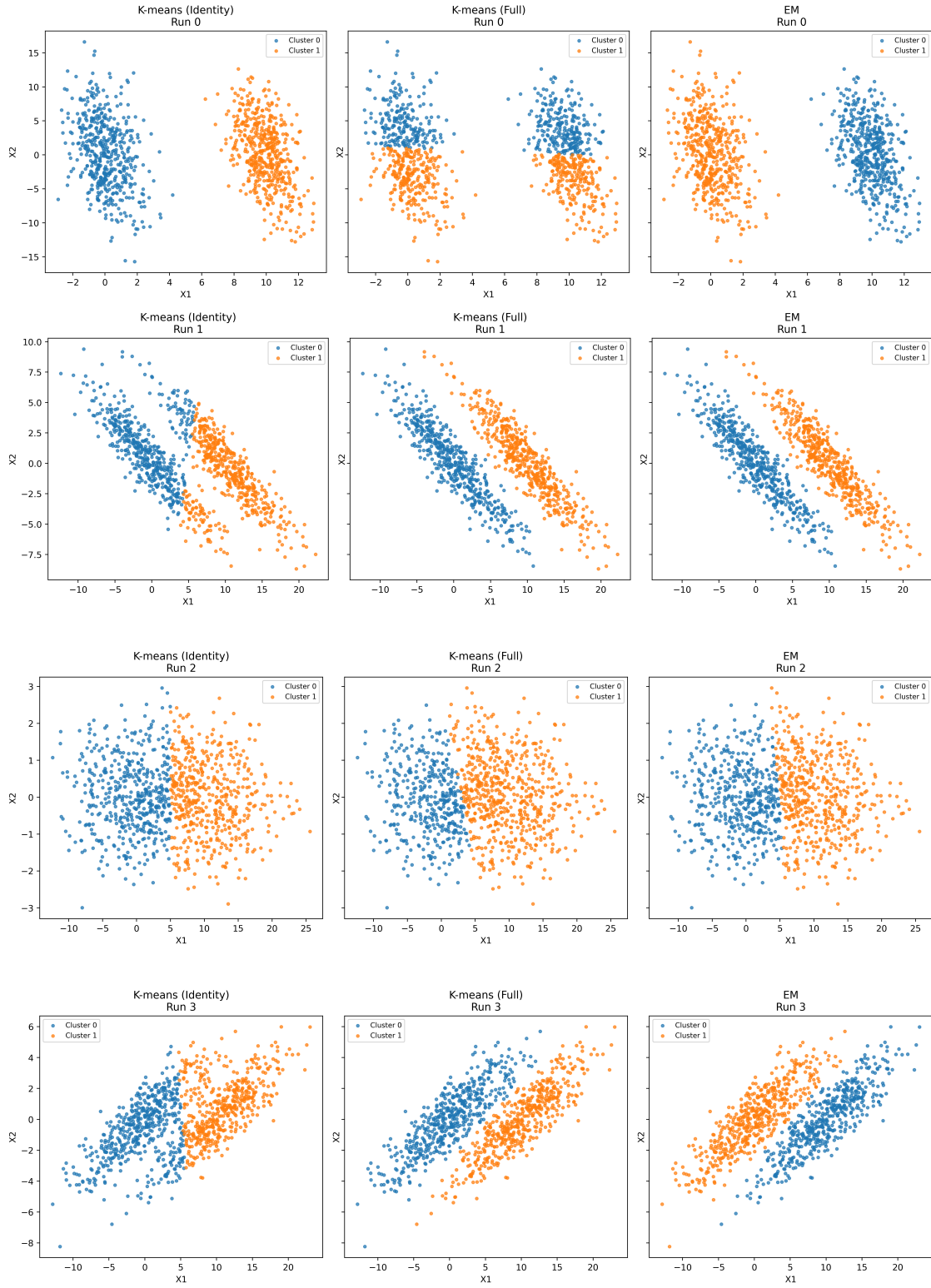


Figure 4: Clustering results for the first four runs.

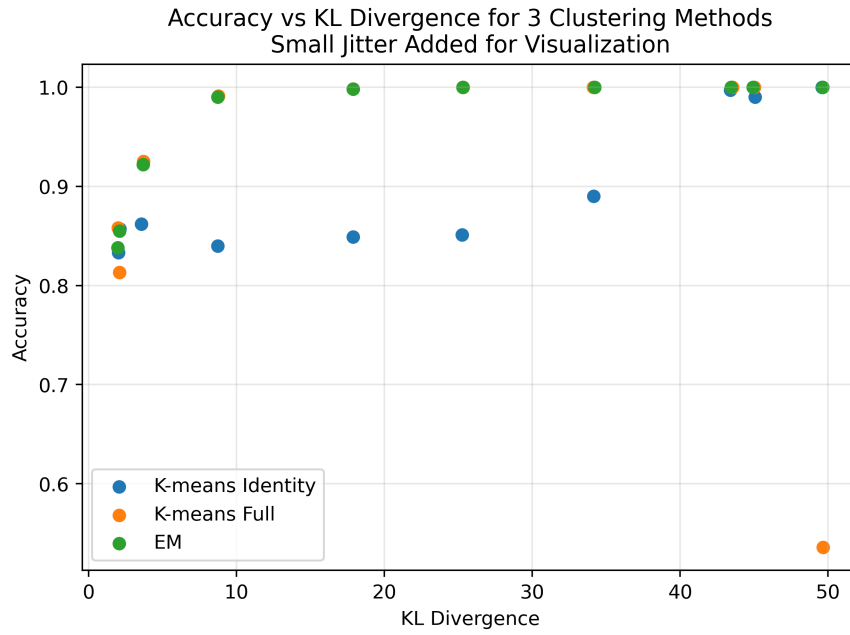


Figure 5: Accuracy vs. KL Divergence for the 3 methods on the 10 runs. Small jitter added to x values for visualization.

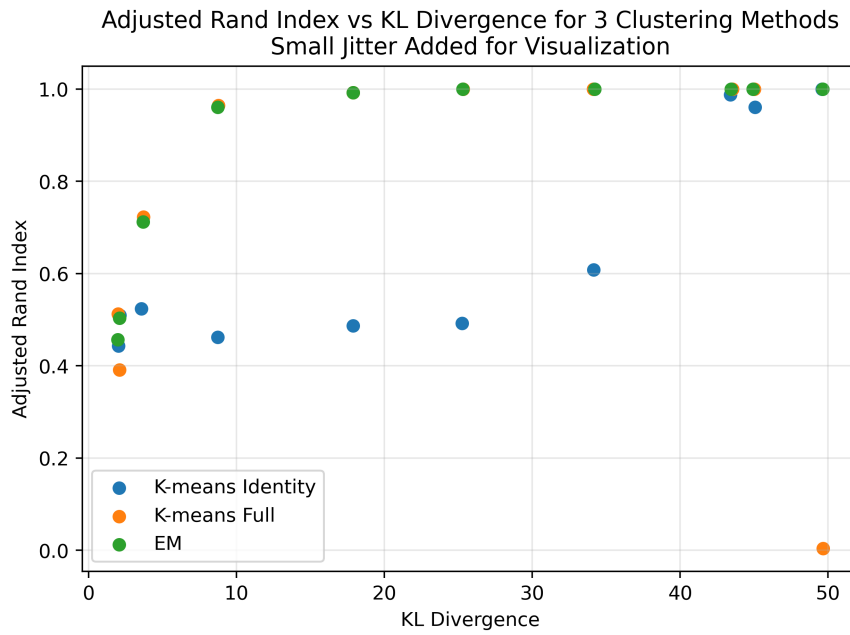


Figure 6: Adjusted Rand Index vs. KL Divergence for the 3 methods on the 10 runs. Small jitter added to x values for visualization.

Run	KL	Acc KM (Id)	ARI KM (Id)	Acc KM (Full)	ARI KM (Full)	Acc EM	ARI EM
0	49.662	1.000	1.000	0.536	0.004	1.000	1.000
1	17.821	0.849	0.487	0.998	0.992	0.998	0.992
2	2.026	0.833	0.443	0.813	0.391	0.838	0.456
3	8.716	0.840	0.462	0.991	0.964	0.990	0.960
4	2.000	0.857	0.509	0.858	0.512	0.855	0.504
5	25.273	0.851	0.492	1.000	1.000	1.000	1.000
6	44.941	0.990	0.960	1.000	1.000	1.000	1.000
7	34.212	0.890	0.608	1.000	1.000	1.000	1.000
8	43.471	0.997	0.988	1.000	1.000	1.000	1.000
9	3.636	0.862	0.524	0.925	0.722	0.922	0.712

Table 1: KL Divergence, Accuracy, and Adjusted Rand Index for each method. Each row is the result from 1 run.

Code

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.stats import multivariate_normal
5 from sklearn.cluster import KMeans
6 from sklearn.metrics.cluster import contingency_matrix
7 from scipy.optimize import linear_sum_assignment
8 from sklearn.metrics import adjusted_rand_score
9 from sklearn.mixture import GaussianMixture
10
11 #functions to impliment the various accuracy measures
12 #https://scikit-learn.org/stable/modules/generated/sklearn.metrics.cluster.contingency_matrix.html
13 #https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html
14 def conting_matrix(observed, predicted):
15     conting_matrix_results = contingency_matrix(observed, predicted)
16
17     #need to maximize, so we need to minimize the negative of the contingency
matrix.
18     #returns the row and column indices for the optimal assignment.
19     row_ind, col_ind = linear_sum_assignment(-conting_matrix_results)
20
21     #maximum diag sum/total number of samples
22     accuracy = conting_matrix_results[row_ind,
23     col_ind].sum()/np.sum(conting_matrix_results)
24
25     return accuracy
26
27 #function to apply the adjusted rand index to the observed and predicted labels
28 #https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html
29 def applying_rand_index(observed, predicted):
30     results = adjusted_rand_score(observed, predicted)
31
32     return results
33
34 #need function for KL divergence between two Gaussians
35 #I think there may be an extra 2 in the KL formula provided
36 #https://mr-easy.github.io/2020-04-16-kl-divergence-between-2-gaussian-distributions/
37 def kl_divergence_gaussians(mu1, cov1, mu2, cov2):
38     #get dimension
39     d = mu1.shape[0]
40
41     #need the log determinant of the covariance matrices
42     #https://numpy.org/doc/2.1/reference/generated/numpy.linalg.slogdet.html
43     sign1, logdet1 = np.linalg.slogdet(cov1)
44     sign2, logdet2 = np.linalg.slogdet(cov2)
```

```

44
45     #trace term
46     t_term = np.trace(np.linalg.inv(cov2) @ cov1)
47
48     #quadtratic term
49     q_term = (mu2 - mu1).T @ np.linalg.inv(cov2) @ (mu2 - mu1)
50
51     #two 1/2 terms. make sure I have everything
52     kl_div = 0.5 * ((logdet2 - logdet1 - d) + t_term + q_term)
53
54     return kl_div
55
56 #part a
57 def make_dataset_part_a(a):
58     XQ = multivariate_normal.rvs(
59         mean=[0.0, 0.0],
60         cov=[[9.0, 0.0], [0.0, 9.0]],
61         size=500
62     )
63     YQ = np.zeros(500)
64
65     mu_a = np.array([a, 0.0])
66     XA = multivariate_normal.rvs(
67         mean=mu_a,
68         cov=np.eye(2),
69         size=500
70     )
71     YA = np.ones(500)
72
73     #merge
74     X = np.vstack([XQ, XA])
75     y = np.concatenate([YQ, YA])
76
77     return X, y
78
79 #https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
80 #https://python-course.eu/machine-learning/expectation-maximization-and-gaussian-mixture-model
81 #https://scikit-learn.org/stable/modules/generated/sklearn.mixture.GaussianMixture.html#sklearn
82
83 #need a way to store the accuracy of each run for both K-means and EM
84 acc_kmeans_list = []
85 acc_em_list = []
86
87 rand_kmeans_list = []
88 rand_em_list = []
89
90 a_list = []

```

```

91 random_run = np.random.randint(0, 10)
92
93
94 #loop over the different a values
95 #just using the default K-means implementation from sklearn for this part
96 for a in range(5):
97     for run in range(10):
98         #get dataset
99         X, y = make_dataset_part_a(a=a)
100
101         #K-means clustering. Adding a random seed this time. for each run and
102         each dataset
103         kmeans = KMeans(n_clusters=2, n_init=1, random_state=1000+a+run)
104         kmeans_labels = kmeans.fit_predict(X)
105
106         #EM clustering
107         EM_cluster = GaussianMixture(n_components=2, n_init=1,
108         random_state=1000+a+run)
109         EM_cluster_labels = EM_cluster.fit_predict(X)
110
111         #computing accuracy
112         accuracy_kmeans = conting_matrix(y, kmeans_labels)
113         accuracy_EM = conting_matrix(y, EM_cluster_labels)
114
115         #append to lists
116         acc_kmeans_list.append(accuracy_kmeans)
117         acc_em_list.append(accuracy_EM)
118
119         #the adjusted rand index
120         rand_index_kmeans = applying_rand_index(y, kmeans_labels)
121         rand_index_EM = applying_rand_index(y, EM_cluster_labels)
122
123         #append the rand_index to the list
124         rand_kmeans_list.append(rand_index_kmeans)
125         rand_em_list.append(rand_index_EM)
126
127         a_list.append(a)
128
129         #plot the clustering results if a==0 and some random run number. 9 here.
130
131         #https://matplotlib.org/stable/gallery/subplots_axes_and_figures/subplots_demo.html
132         if a == 0 and run == random_run:
133             #set up figure
134             fig, axes = plt.subplots(1, 2, figsize=(12,5))
135
136             #K-means plot
137             axes[0].scatter(X[:, 0], X[:, 1],

```



```

135         c=kmeans_labels,
136         s=10,alpha=0.7
137     )
138     axes[0].set_title(f"K-means_Clustering_(a={a},run={run})")
139     axes[0].set_xlabel("X1")
140     axes[0].set_ylabel("X2")
141
142     #EM plot
143     axes[1].scatter(X[:, 0], X[:, 1],
144                     c=EM_cluster_labels,
145                     s=10, alpha=0.7
146                 )
147     axes[1].set_title(f"EM_Clustering_(a={a},run={run})")
148     axes[1].set_xlabel("X1")
149     axes[1].set_ylabel("X2")
150
151     plt.tight_layout()
152     plt.savefig("assignments/hw08/figures/cluster_results_a.png", dpi=400,
153                 bbox_inches="tight")
154     plt.close()
155     #plt.show()
156
157     #little hard to see, maybe add a jitter?
158     #plot the accuracy of each run vs as a seperate dots on the same plot
159     #different colors for K-means and EM
160     plt.figure()
161     plt.scatter(a_list, np.array(acc_kmeans_list), label = "K-means")
162     plt.scatter(a_list, np.array(acc_em_list), label = "EM")
163     plt.xlabel("Dataset_a")
164     plt.ylabel("Accuracy")
165     plt.title("Accuracy_across_5_datasets,_10_runs_each")
166     plt.legend()
167     plt.grid(alpha=0.3)
168     plt.tight_layout()
169     plt.savefig("assignments/hw08/figures/accuracy_vs_a.png", dpi=400,
170                 bbox_inches="tight")
171     plt.close()
172     #plt.show()
173
174     #plotting the adjusted rand index now. Same as above but with the rand index
175     instead of accuracy
176     plt.figure()
177     plt.scatter(a_list, np.array(rand_kmeans_list), label = "K-means")
178     plt.scatter(a_list, np.array(rand_em_list), label = "EM")
179     plt.xlabel("Dataset_a")
180     plt.ylabel("Adjusted_Rand_Index")
181     plt.title("Adjusted_Rand_Index_across_5_datasets,_10_runs_each")

```

```

179 plt.legend()
180 plt.grid(alpha=0.3)
181 plt.tight_layout()
182 plt.savefig("assignments/hw08/figures/ari_vs_a.png", dpi=400,
    bbox_inches="tight")
183 plt.close()
184 #plt.show()
185
186 #function to impliment k-means clustering from the slides, which cares about the
    covariance structure
187 #k-means with either no covariance modelling or full covariance modelling
188 def k_means_clustering(X, n_clusters, max_iters=100, seed=123,
189     covariance="identity"):
190
191     #set random seed and get dimensions
192     np.random.seed(seed)
193     n, d = X.shape
194
195     #get initial centers by randomly sampling from the data points without
    replacement
196
197     #https://numpy.org/doc/stable/reference/random/generated/numpy.random.choice.html
    idx = np.random.choice(n, n_clusters, replace=False)
198     centers = X[idx]
199
200     #initialize covariance matrices and labels. Set labels to -1 so they are
    not initialized to any cluster
201     Sigmas = np.tile(np.eye(d), (n_clusters, 1, 1))
202     labels = -np.ones(n, dtype=int)
203
204     for i in range(max_iters):
205         old_labels = labels.copy()
206
207         #compute distances
208         dists = np.zeros((n, n_clusters))
209
210         #if branch for covariance type.
211         if covariance == "identity":
212             for k in range(n_clusters):
213                 dists[:, k] = np.linalg.norm(X - centers[k], axis=1)**2
214
215         elif covariance == "full":
216             for k in range(n_clusters):
217                 diff = X - centers[k]
218                 invS = np.linalg.inv(Sigmas[k])
219                 dists[:, k] = np.sum(diff @ invS * diff, axis=1)
220

```

```

221         #assign clusters
222         #https://numpy.org/devdocs/reference/generated/numpy.argmin.html
223         labels = np.argmin(dists, axis=1)
224
225         #stop if labels don't change, even if we haven't hit max_iters
226         if np.array_equal(labels, old_labels):
227             break
228
229         #update centers and covariances for each group
230         for k in range(n_clusters):
231
232             #select the points in the current cluster
233             cluster_points = X[labels == k]
234
235             #compute the new mean/centroid
236             centers[k] = cluster_points.mean(axis=0)
237
238             #compute the covariance matrix for the cluster if we are using full
239             covariance
240             if covariance == "full":
241                 Sigmas[k] = np.cov(cluster_points, rowvar=False)
242
243             #otherwise, just use an identity matrix
244             else:
245                 Sigmas[k] = np.eye(d)
246
247         return labels, centers, Sigmas
248
249 #part b
250 results = []
251 plot_runs = []
252
253 for i in range(10):
254     #each element is from a standard normal distribution
255
256     #https://numpy.org/doc/2.1/reference/random/generated/numpy.random.normal.html
257     M = np.random.normal(0, 1, size=(2, 2))
258
259     #SVD decomposition
260     #U rotates the data
261     U, S, Vh = np.linalg.svd(M)
262
263     #create Sigma matrix from U
264     #each run produces a rotated matrix with different variances along the two
265     dimensions
266     D = np.diag([25,1])
267     Sigma = U @ D @ U.T

```

```

265 #generate X centered at 0 with covariance Sigma plus labels
266 XQ = multivariate_normal.rvs(
267     mean=[0.0, 0.0],
268     cov=Sigma,
269     size=500
270 )
271 YQ = np.zeros(500)
272
273 #generate XP from (10,0) with covariance Sigma
274 XP = multivariate_normal.rvs(
275     mean=[10.0, 0.0],
276     cov=Sigma,
277     size=500
278 )
279 YP = np.ones(500)
280
281 #stack everything together
282 X = np.vstack([XQ, XP])
283 y = np.concatenate([YQ, YP])
284
285 #compute the KL divergence between the two distributions
286 kl = kl_divergence_gaussians(
287     mu1=np.array([10.0,0.0]),
288     cov1=Sigma,
289     mu2=np.array([0.0,0.0]),
290     cov2=Sigma
291 )
292
293 #K means with no covariance modelling (Identity)
294 k_mean_identity_labels, centers_identity, Sigmas_identity =
295 k_means_clustering(X, n_clusters=2, covariance="identity", seed=1000+i)
296
297 #K means with full covariance matrices
298 k_means_full_labels, centers_full, Sigmas_full = k_means_clustering(X,
299 n_clusters=2, covariance="full", seed=1000+i)
300
301 #EM clustering. Gaussian Mixture implements EM algorithm.
302 #https://scikit-learn.org/stable/modules/mixture.html#gmm
303 EM_cluster = GaussianMixture(n_components=2, n_init=1, random_state=1000+i)
304 EM_cluster_labels = EM_cluster.fit_predict(X)
305
306 #store what we need to plot the three clustering methods for the first four
runs
307 if i < 4:
308     plot_runs.append({
309         "run": i,

```

```

309         "X": X,
310         "y_true": y,
311         "pred_id": k_mean_identity_labels,
312         "pred_full": k_means_full_labels,
313         "pred_em": EM_cluster_labels,
314     })
315
316     #adding everything to the results list
317     results.append({
318         "run": i,
319         "KL_divergence": kl,
320         "accuracy_kmeans_id": conting_matrix(y, k_mean_identity_labels),
321         "ARI_kmeans_id": adjusted_rand_score(y, k_mean_identity_labels),
322         "accuracy_kmeans_full": conting_matrix(y, k_means_full_labels),
323         "ARI_kmeans_full": adjusted_rand_score(y, k_means_full_labels),
324         "accuracy_EM": conting_matrix(y, EM_cluster_labels),
325         "ARI_EM": adjusted_rand_score(y, EM_cluster_labels),
326     })
327
328
329     #everything after this is just plotting stuff.
330     #plotting the clustering results for the first four runs.
331     for item in plot_runs:
332
333         run = item["run"]
334         X = item["X"]
335
336         #share the same axes for all 3 methods
337         fig, axes = plt.subplots(1, 3, figsize=(15,5), sharex=True, sharey=True)
338
339         #list out the titles, and labels for the three methods
340         titles = ["K-means_Identity", "K-means_Full", "EM"]
341         label_sets = [item["pred_id"], item["pred_full"], item["pred_em"]]
342
343         #found this zip function. Pretty nice. Iterates through axes, labels, and
titles.
344         #https://www.codecademy.com/article/python-zip-function
345         for ax, labels, title in zip(axes, label_sets, titles):
346
347             for k in np.unique(labels):
348                 ax.scatter(X[labels==k,0], X[labels==k,1], s=10, alpha=0.7,
349                    label=f"Cluster_{k}")
350
351             ax.set_title(f"{title}\nRun_{run}")
352             ax.set_xlabel("X1")
353             ax.set_ylabel("X2")
354             ax.legend(fontsize=8)

```

```

354     plt.tight_layout()
355     plt.savefig(f"assignments/hw08/figures/partb_run{run}_clusters.png",
356     dpi=400, bbox_inches="tight")
357     plt.close()
358
359
360 #plotting the accuracy bs KL divergence for the three methods across all 10 runs.
361 results_df = pd.DataFrame(results)
362
363 #add some jitter to see things better
364 jitter = 0.05
365 KL_id = results_df["KL_divergence"] + np.random.normal(0, jitter,
366     size=len(results_df))
367 KL_full = results_df["KL_divergence"] + np.random.normal(0, jitter,
368     size=len(results_df))
369 KL_em = results_df["KL_divergence"] + np.random.normal(0, jitter,
370     size=len(results_df))
371
372 plt.figure()
373 plt.scatter(KL_id, results_df["accuracy_kmeans_id"], label="K-means_Identity")
374 plt.scatter(KL_full, results_df["accuracy_kmeans_full"], label="K-means_Full")
375 plt.scatter(KL_em, results_df["accuracy_EM"], label="EM")
376 plt.xlabel("KL_Divergence")
377 plt.ylabel("Accuracy")
378 plt.title("Accuracy_vs_KL_Divergence_for_3_Clustering_Methods\nSmall_Jitter_
379     Added_for_Visualization")
380 plt.legend()
381 plt.grid(alpha=0.3)
382 plt.tight_layout()
383 plt.savefig("assignments/hw08/figures/KL_vs_Accuracy.png", dpi=400,
384     bbox_inches="tight")
385 plt.close()
386 #plt.show()
387
388 #plotting the Adjusted Rand Index vs KL divergence for the three methods across
389 all 10 runs
390 plt.figure()
391 plt.scatter(KL_id, results_df["ARI_kmeans_id"], label="K-means_Identity")
392 plt.scatter(KL_full, results_df["ARI_kmeans_full"], label="K-means_Full")
393 plt.scatter(KL_em, results_df["ARI_EM"], label="EM")
394 plt.xlabel("KL_Divergence")
395 plt.ylabel("Adjusted_Rand_Index")
396 plt.title("Adjusted_Rand_Index_vs_KL_Divergence_for_3_Clustering_Methods\nSmall_
397     Jitter_Added_for_Visualization")
398 plt.legend()
399 plt.grid(alpha=0.3)

```

```

393 plt.tight_layout()
394 plt.savefig("assignments/hw08/figures/KL_vs_ARI.png", dpi=400,
395             bbox_inches="tight")
396 plt.close()
397
398 #reporting the table and reordering the columns
399 df = results_df[[
400     "run",
401     "KL_divergence",
402     "accuracy_kmeans_id",
403     "ARI_kmeans_id",
404     "accuracy_kmeans_full",
405     "ARI_kmeans_full",
406     "accuracy_EM",
407     "ARI_EM"
408 ]]
409
410 #rename the columns
411 df.columns = [
412     "Run",
413     "KL",
414     "Acc_KM_Id",
415     "ARI_KM_Id",
416     "Acc_KM_Full",
417     "ARI_KM_Full",
418     "Acc_EM",
419     "ARI_EM"
420 ]
421
422 #need to send this table to LATEX
423 LATEX_table = df.to_latex(
424     index=False,
425     caption=None,
426     label=None,
427     column_format="c" * df.shape[1],
428     escape=True,
429     float_format="%.3f"
430 )
431
432 #save latex table
433 with open("assignments/hw08/output/final_table.tex", "w") as f:
434     f.write(LATEX_table)

```